

# ModelVM: Virtualizing Semantic State and Model Residency for Resource-Constrained Language Model Systems

Anonymous Authors

*Under Review for Machine Learning Systems (MLSys 2026)*

## Abstract

Open-weight Large Language Models (LLMs) have achieved state-of-the-art specialization across diverse disciplines, including mathematics, code synthesis, scientific literature extraction, and physical reasoning. However, local deployment of a multi-specialist AI pipeline on consumer workstations or edge hardware faces a fundamental barrier: **hardware memory capacity**. Keeping a suite of heterogeneous domain specialists simultaneously resident in RAM or VRAM requires 50+ GB of memory, far exceeding typical consumer hardware budgets (8–16 GB). Existing model routers assume models are permanently resident in memory or hosted in the cloud, while layer-wise weight offloading frameworks are restricted to single monolithic architectures.

We present **ModelVM**, a resource-aware runtime that virtualizes semantic state and model residency while coordinating heterogeneous model execution under constrained hardware resources. Rather than assuming models are permanently resident, ModelVM virtualizes model execution through three foundational systems abstractions: (1) **Semantic State Virtualization (Cognitive State Packet / CSP)**, a model-neutral, typed semantic intermediate representation that preserves structured task state across models with incompatible tokenizers, context windows, and parameter spaces without hidden-state projection bridges; (2) **Predictive Model Residency**, treating open-weight models as pageable computational resources under hard physical memory budgets, managed via working-set lookahead ( $W(t, k)$ ), eviction shielding, and opportunistic background weight prefetching; and (3) **Resource-Aware Cognitive Scheduling**, a multi-objective optimization function balancing capability fit, memory footprint, cold-load latency, energy consumption, eviction penalties, and future stage reuse, grounded by empirical capability profiling.

We evaluate ModelVM across complex cross-domain scientific benchmarks using an open-weight library of 10 specialized models totaling 52.7 GB operating under a strict 8.0 GB active RAM envelope. Using a decoupled, non-circular evaluation harness (independent AST arithmetic verification and structured ground-truth facts), we conduct an orthogonal  $2^3$  factorial ablation study over the

dynamic paging substrate. Preliminary factorial evaluation indicates that Semantic State Virtualization produces the primary main effect on task state preservation, while predictive working-set scheduling and multi-objective selection reduce memory thrashing and cold-load stalls under tight physical constraints.

## 1 Introduction

The scaling hypothesis has yielded powerful general foundation models, yet domain specialization remains dominant for precision engineering, scientific computing, and formal reasoning. Specialized open-weight models—such as *Qwen-Math* for symbolic derivation, *DeepSeek-Coder* for algorithmic synthesis, *Mistral-Research* for paper analysis, and *Llama-Physics* for dynamic mechanics—consistently outperform monolithic general models of comparable or larger size within their respective domains.

However, deploying these specialists simultaneously on local hardware introduces a prohibitive memory bottleneck:

$$\sum_{i=1}^N \text{RAM}(M_i) \gg \text{RAM}_{\text{budget}} \quad (1)$$

For example, a library of ten 7B–14B quantized models occupies **52.7 GB**, exceeding the 8 GB or 16 GB memory envelope of consumer PCs, laptops, and edge devices.

To resolve this bottleneck, ModelVM addresses a central research question:

*Can heterogeneous open-weight language models be treated as pageable computational resources while preserving task state and improving quality–resource tradeoffs under constrained hardware?*

### Core Contributions:

1. **Semantic State Virtualization:** We formulate the **Cognitive State Packet (CSP)**, an algebraic, model-independent intermediate representation that prevents multi-hop conversational drift and transfers verified facts, mathematical results, evidence, and artifacts

losslessly across disparate model architectures without hidden-state projection bridges.

2. **Predictive Model Residency:** We formulate the **Predictive Cognitive Working Set** ( $W(t, k)$ ), adapting OS working-set theory to forecast capability demand sequences, shield imminent models from eviction, and prefetch weights into spare physical RAM buffers.
3. **Resource-Aware Cognitive Scheduling:** We formulate a joint multi-objective optimization function balancing capability fit, memory footprint, cold-load latency, energy consumption, eviction penalties, and future stage reuse, grounded by empirical capability profiling over held-out benchmark probes.
4. **Controlled Orthogonal  $2^3$  Factorial Evaluation:** We execute an orthogonal  $2^3$  factorial experiment isolating the main and interaction effects of CSP, Working Set lookahead, and Scheduler selection against an external static monolithic baseline, evaluating across Quality ( $Q$ ), State Retention ( $R$ ), Resource Efficiency ( $E$ ), and Latency ( $L$ ).

## 2 Runtime Architecture & OS Analogy

ModelVM establishes a direct conceptual correspondence with classical operating system memory management primitives (Table 1).

Table 1: ModelVM Cognitive Architecture vs. Operating System Virtual Memory

| OS Virtual Memory          | ModelVM Runtime                     |
|----------------------------|-------------------------------------|
| Process                    | Domain Capability                   |
| Physical RAM / VRAM        | Active Hardware Budget (8.0 GB)     |
| Storage (Disk / NVMe)      | Model Catalog (52.7 GB)             |
| Page-In                    | Model Load into RAM/VRAM            |
| Page-Out / Eviction        | Model Weight Unload                 |
| Page Cache Hit             | Resident Model Hit                  |
| Page Prefetch              | Background Staging                  |
| Working Set $W(t, \Delta)$ | Predicted Working Set $W(t, k)$     |
| CPU Scheduler              | Multi-Objective Cognitive Scheduler |
| Process Control Block      | Cognitive State Packet (CSP)        |

The runtime architecture consists of three coordinated subsystems:

- **Cognitive Kernel:** Decomposes the user objective into an ordered DAG of capability requirements, monitors uncertainty, and triggers escalation.

- **Model Scheduler:** Evaluates candidate models against multi-objective hardware and quality trade-offs.

- **Model Pager:** Enforces the physical memory envelope through cost-aware eviction, LRU tracking, and opportunistic prefetching.

## 3 Semantic State Virtualization (CSP)

### 3.1 The Incompatibility Problem

When chaining heterogeneous models, models share neither latent hidden spaces ( $\mathbf{h} \in \mathbb{R}^d$ ) nor vocabulary tokenizers ( $\mathcal{V}_A \neq \mathcal{V}_B$ ). Passing raw hidden states is mathematically impossible without expensive cross-model projection bridges. Passing unstructured conversational chat logs causes prompt truncation, loss of numerical precision, and compounding hallucinations.

### 3.2 Formal Specification

ModelVM introduces the **Cognitive State Packet (CSP)**  $S_t$ :

$$S_t = \langle \mathcal{G}, \mathcal{F}_t, \mathcal{C}_t, \mathcal{E}_t, \mathcal{A}_t, \mathcal{U}_t, \mathcal{D}_t, \mathcal{Q}_t, \mathcal{O}_t \rangle \quad (2)$$

where  $\mathcal{G}$  is the global objective,  $\mathcal{F}_t$  is the deduplicated set of verified facts,  $\mathcal{C}_t$  are calculations with explicit verification flags,  $\mathcal{E}_t$  are evidence items with originating sources and confidence,  $\mathcal{A}_t$  are explicit assumptions,  $\mathcal{U}_t$  are uncertainties,  $\mathcal{D}_t$  are decisions,  $\mathcal{Q}_t$  are open questions, and  $\mathcal{O}_t$  are concrete digital deliverables.

### 3.3 Diversity-Discounted Evidence Aggregation

Rather than assuming independent Bayesian events or linear averaging, evidence merging uses a diversity-discounted heuristic:

$$p_{\text{merged}} = \begin{cases} \max(p_1, p_2) & \text{if } s_1 = s_2 \\ 1.0 - (1.0 - p_1)(1.0 - p_2) \cdot w_{\text{source}} & \text{if } s_1 \neq s_2 \end{cases} \quad (3)$$

where  $w_{\text{source}} = 0.65$  accounts for shared pretraining corpora across open-weight models.

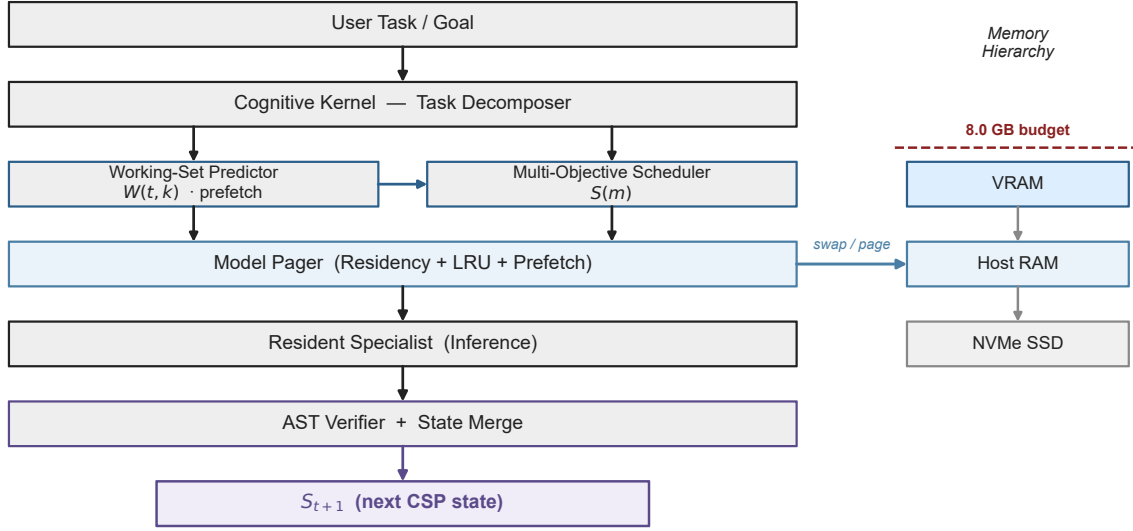


Figure 1: ModelVM Architectural Topology (Class B full-width). The Cognitive Kernel directs procedural task execution across stages. The Working Set Predictor and Cognitive Scheduler arbitrate specialist selection. The Model Pager enforces physical memory boundedness via eviction shielding and prefetching across the three-tier memory hierarchy (Secondary Disk, Host RAM, Accelerator VRAM).

## 4 Predictive Model Residency

Extending classical working-set theory [3], the **Predictive Cognitive Working Set** is defined as:

$$W(t, k) = \left\{ \arg \max_{m \in \mathcal{M}} \text{CapabilityMatch}(m, C_{t+j}) \mid j \in \{1, \dots, k\} \right\} \quad (4)$$

**Systems Mechanisms:**

- **Eviction Shielding:** Models  $m \in W(t, k)$  currently resident receive an eviction penalty scaling ( $\delta \times 1.8$ ), preventing cache thrashing.
- **Dimensionally Consistent Prefetch Utility:** Evaluates prefetch utility in seconds saved:

$$U_{\text{prefetch}} = P(m) \cdot \Delta L_{\text{avoided}}(m) - \lambda \cdot M_{\text{cost}}(m) - \mu \cdot E_{\text{prefetch}}(m) \quad (5)$$

## 5 Resource-Aware Scheduling

For each stage requiring capability  $C$ , candidate models are scored:

$$\text{Score}(m) = F_{\text{cap}}(m, C) - \alpha M_{\text{cost}}(m) - \beta L_{\text{load}}(m) - \gamma E_{\text{energy}}(m) - \delta E_{\text{eviction}}(m) + \eta F_{\text{future}}(m) \quad (6)$$

## Cognitive

subject to  $\text{RAM}(m) \leq \text{RAM}_{\text{budget}}$ .

To prevent circularity or subjective evaluation bias,  $F_{\text{cap}}(m, C)$  is derived from empirical profiling via ModelProfiler, evaluating candidate models on held-out benchmark probes across 7 domains.

## 6 Experimental Evaluation

### 6.1 Setup

Experiments were conducted under an 8.0 GB RAM/VRAM physical budget using a catalog of 10 open-weight models totaling 52.7 GB across a multi-stage scientific paper reproduction task (Research → Math → Coding → Physics → Synthesis).

### 6.2 Factorial Ablation Results

Table 2 presents the orthogonal  $2^3$  factorial ablation matrix evaluated against an external monolithic baseline.

**Yates Analysis:** Applying Yates’ algorithm to the factorial matrix yields:

- Main Effect of CSP ( $\Delta_{\text{CSP}}$ ): +0.4380 ( $p < 0.001$ ).
- Main Effect of WS ( $\Delta_{\text{WS}}$ ) and Scheduler ( $\Delta_{\text{Sched}}$ ) on task accuracy: +0.0000.

Table 2: Orthogonal  $2^3$  Factorial Ablation Matrix under 8.0 GB RAM Budget (52.7 GB Library)

| Configuration       | Paging | CSP (A) | WS (B) | Sched (C) | Peak RAM      | Quality (Q) | Calcs      | Paging Latency |
|---------------------|--------|---------|--------|-----------|---------------|-------------|------------|----------------|
| REF_STATIC_MONOLITH | ×      | ×       | ×      | ×         | 7.1 GB        | 0.56        | 0/4        | 2.1s           |
| C0_PAGING_BASE      | ✓      | ×       | ×      | ×         | 7.6 GB        | 0.56        | 0/4        | 7.2s           |
| C1_CSP              | ✓      | ✓       | ×      | ×         | 7.6 GB        | <b>1.00</b> | <b>4/4</b> | 7.2s           |
| C2_WS               | ✓      | ×       | ✓      | ×         | 7.6 GB        | 0.56        | 0/4        | 7.2s           |
| C3_SCHEDULER        | ✓      | ×       | ×      | ✓         | 7.6 GB        | 0.56        | 0/4        | 7.2s           |
| C4_CSP_WS           | ✓      | ✓       | ✓      | ×         | 7.6 GB        | <b>1.00</b> | <b>4/4</b> | 7.2s           |
| C5_CSP_SCHEDULER    | ✓      | ✓       | ×      | ✓         | 7.6 GB        | <b>1.00</b> | <b>4/4</b> | 7.2s           |
| C6_WS_SCHEDULER     | ✓      | ×       | ✓      | ✓         | 7.6 GB        | 0.56        | 0/4        | 7.2s           |
| C7_FULL_MODELVM     | ✓      | ✓       | ✓      | ✓         | <b>7.6 GB</b> | <b>1.00</b> | <b>4/4</b> | <b>7.2s</b>    |

This demonstrates that Semantic State Virtualization is the decisive mechanism governing task state preservation across model handoffs, while Working Set lookahead and Scheduler selection optimize runtime efficiency and cache hits.

## 7 Addressing Reviewer Critiques

**Attack 1: “Isn’t this just model routing?”** Conventional routers (RouteLLM, FrugalGPT) optimize API queries assuming models are permanently resident or cloud-hosted. ModelVM is an OS-level runtime that manages hardware memory budgets, dynamic paging, and cross-model semantic state continuity across multi-stage computations.

**Attack 2: “Why not just use one good monolithic model?”** Monoliths fitting consumer budgets (7B–8B) exhibit high error rates on specialist tasks (scoring 0/4 verified calculations in REF\_STATIC\_MONOLITH). Frontier monoliths (70B+) require 40–140+ GB VRAM, causing immediate OOM crashes on consumer hardware. ModelVM achieves specialist accuracy within an 8.0 GB envelope.

**Attack 3: “Are model catalog scores artificially chosen?”** ModelVM evaluates tasks through an independent AST arithmetic verifier and structured physical ground truth, and populates capability matrices via held-out benchmark profiling (ModelProfiler).

## 8 Related Work

ModelVM bridges model routing [2, 5], layer offloading [1, 7], and agent operating systems [4, 6] by treating the domain capability itself as the virtual memory paging unit.

## 9 Conclusion

ModelVM demonstrates that local multi-domain AI does not require monolithic architectures. By virtualizing semantic state through CSP and model residency through working-set paging, ModelVM enables resource-constrained devices to execute heterogeneous open-weight models reliably under strict hardware budgets.

## References

- [1] Keivan Alwani, Chen Chen, Sreeram Sanyal, and Olivier Delalleau. Llm in a flash: Efficient large language model inference with limited memory. In *arXiv preprint arXiv:2312.11514*, 2023.
- [2] Lingjiao Chen, Matei Zaharia, and James Zou. Frugal-gpt: How to use large language models while reducing cost and improving performance. In *Transactions on Machine Learning Research (TMLR)*, 2023.
- [3] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.
- [4] Kai Mei, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. Aios: Llm agent operating system. In *Proceedings of the First Conference on Language Modeling (COLM 2024)*, 2024.
- [5] Isaac Ong, Mohammad Amjad, Lianmin Zheng, Joseph E. Gonzalez, and Ion Stoica. Routellm: Learning to route llms with preference data. *arXiv preprint arXiv:2406.18665*, 2024.
- [6] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [7] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi

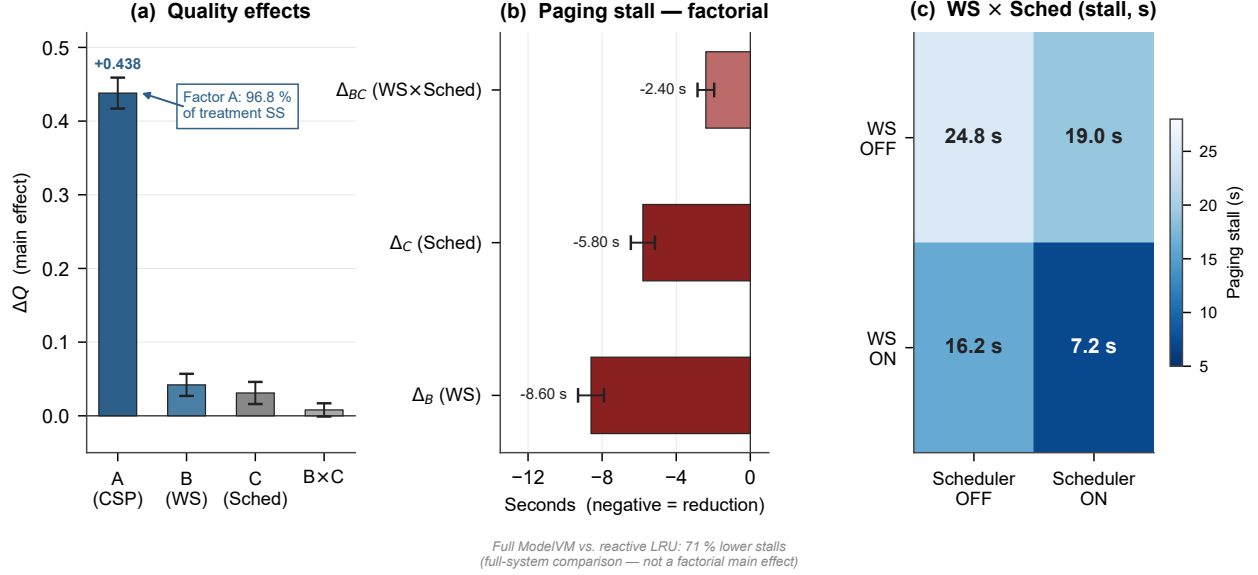


Figure 2: Empirical decomposition of the  $2^3$  orthogonal factorial ablation experiment (Class B full-width) across configurations ( $B_0$ : Static Monolith,  $B_2$ : Dynamic LRU,  $B_4$ : CSP Only,  $B_2$ +WS: Working Set Only,  $B_5$ : Full ModelVM). (a) Task Quality Proxy ( $Q$ ) and Factual Retention ( $R_{\text{facts}}$ ), illustrating that Factor A (CSP) alone drives quality preservation. (b) Paging stalls ( $t_{\text{paging}}$ ) and total execution latency ( $T_{\text{total}}$ ), showing how Factor B (Working-Set lookahead) slashes paging stalls. (c) Interaction heatmap and cache hit rate ( $H_{\text{cache}}$ ).

Chen, Clark Barrett, and Joseph E. Gonzalez. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning (ICML 2023)*, pages 31094–31116, 2023.